Reviewer Pack — Schur-Weyl Decomposition Irreducible Component Gap for Perma…

Entry 386190a1e55e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 21:30:36 UTC

Schur-Weyl Decomposition Irreducible Component Gap for Permanent vs Determinant

Entry ID: `386190a1e55e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 21:30:36 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Discrepancy of Communication Matrices

Statement:

> For random n×n matrices A with entries ±1, let λ(A) be the number of irreducible components in the Schur-Weyl decomposition of (A⊗A⊗…⊗A) (n times). Then λ(perm_n) ≥ 2^{Ω(n)} while λ(det_n) = O(n^3), with equality holding for all n ≤ 40.

Rationale (proposer's reasoning):

> The Schur-Weyl decomposition captures symmetry structures inherent to permanent/determinant matrices. Discrepancy bounds for communication matrices of these functions may be tied to the complexity of their representation-theoretic decompositions, as symmetric group actions encode algebraic dependencies.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b69c78a3ea600961`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `INCONCLUSIVE` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Schur-Weyl decomposition discrepancy communication matrices` - `Schur-Weyl decomposition permanent determinant discrepancy` - `Schur-Weyl decomposition irreducible components permanent determinant gap`

Top relevant hits considered: - [http://arxiv.org/abs/1807.04318v2] On the Discrepancy of Random Matrices with Many Columns - [http://arxiv.org/abs/1503.04226v2] Symmetric Hadamard matrices of order 116 and 172 exist - [http://arxiv.org/abs/2403.03513v3] Spectrum of random centrosymmetric matrices; CLT and Circular law - [http://arxiv.org/abs/1503.00822v2] Product Ranks of the 3×3 Determinant and Permanent - [http://arxiv.org/abs/2307.01912v3] Yay for Determinants! - [http://arxiv.org/abs/1610.09552v1] Entanglement distillation using Schur-Weyl decomposition for three qubits - [http://arxiv.org/abs/1809.03683v8] Irreducible components of affine Deligne-Lusztig varieties - [s2:10.4171/OWR/2020/9] Mini-Workshop: Kronecker, Plethysm, and Sylow Branching Coefficients and their Applications to Complexity Theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=12.8s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def hook_length(n, k):
        return (n - k + 1) * (n - k) // 2

    def young_diagram(m, n):
        diag = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            diag[i][i] = hook_length(i, 0)
            for j in range(1, min(i, n) + 1):
                diag[i][j] = diag[i - 1][j - 1] + hook_length(i, j)
        return diag

    def count_irreducible_components(diag):
        m, n = len(diag) - 1, len(diag[0]) - 1
        count = 0
        for i in range(m + 1):
            for j in range(n + 1):
                if diag[i][j] == 1:
                    count += 1
        return count
```

```
def perm_matrix(n):
    matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        matrix[i][(i + 1) % n] = 1
    return matrix

def det_matrix(n):
    matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if (i + j) % 2 == 0:
                matrix[i][j] = 1
    return matrix

def tensor_power(matrix, k):
    result = matrix
    for _ in range(1, k):
        result = [[sum(result[i][m] * matrix[m][j] for m in range(n)) for j in range(n)] for i
    return result

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    perm = perm_matrix(n)
    det = det_matrix(n)

    perm_tensor = tensor_power(perm, n)
    det_tensor = tensor_power(det, n)

    perm_diag = young_diagram(len(perm_tensor), len(perm_tensor[0]))
    det_diag = young_diagram(len(det_tensor), len(det_tensor[0]))

    perm_components = count_irreducible_components(perm_diag)
    det_components = count_irreducible_components(det_diag)

    results.append({
        "n": n,
        "perm_components": perm_components,
        "det_components": det_components
    })

total_perm_components = sum(result["perm_components"] for result in results)
total_det_components = sum(result["det_components"] for result in results)

metric_value = total_perm_components / len(n_values)
conjecture_holds = all(result["perm_components"] >= 2**(n/2) for result in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Irreducible Components",
    "metric_value": metric_value,
    "instances_tested": len(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ping_undefined'}
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
TRIAL: {'metric_name': 'Irreducible Components', 'metric_value': 1.0, 'instances_tested': 6, 'conjec
RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The conjecture is contradicted by multiple trials with identical counterexamples 'mapping_undefined'. | next: Investigate the 'mapping_undefined' counterexample's origin in the Schur-Weyl decomposition computation

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111774
2	propose	ollama_remote	qwen3:8b	0	0	30878
3	preregistration	ollama_remote	qwen3:8b	0	0	24463
4	novelty	ollama_remote	qwen3:8b	0	0	18284
5	novelty	ollama_remote	qwen3:8b	0	0	24667
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10959
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11076
8	critic	ollama_remote	qwen3:8b	0	0	30024
9	judge	ollama_remote	qwen3:8b	0	0	16399

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 278523 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/386190a1e55e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/386190a1e55e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/386190a1e55e.tar.gz` (if generated)